OTHER

Use read as an alternative to the sleep command

Surprisingly, sleep is an external command and not a bash built-in.

CAVEAT: Requires bash 4+

Example Function:

```
read_sleep() {
    # Usage: read_sleep 1
    #        read_sleep 0.2
    read -rt "$1" <> <(:) || :
}
```

Example Usage:

```
read_sleep 1
read_sleep 0.1
read_sleep 30
```

For performance-critical situations, where it is not economic to open and close an excessive number of file descriptors, the allocation of a file descriptor may be done only once for all invocations of read:

(See the generic original implementation at https://blog.dhampir.no/content/sleeping-without-a-subprocess-in-bash-and-how-to-sleep-forever)

```
exec {sleep_fd}<> <(:)
while some_quick_test; do
    # equivalent of sleep 0.001
    read -t 0.001 -u $sleep_fd
done
```

Check if a program is in the user's PATH

```
# There are 3 ways to do this and either one can be used.
type -p executable_name &>/dev/null
hash executable_name &>/dev/null
command -v executable_name &>/dev/null

# As a test.
if type -p executable_name &>/dev/null; then
    # Program is in PATH.
fi

# Inverse.
if ! type -p executable_name &>/dev/null; then
    # Program is not in PATH.
fi

# Example (Exit early if program is not installed).
```

```
if ! type -p convert &>/dev/null; then
    printf '%s\n' "error: convert is not installed, exiting..."
    exit 1
fi
```

Get the current date using `strftime`

Bash's `printf` has a built-in method of getting the date which can be used in place of the `date` command.

CAVEAT: Requires bash 4+

Example Function:

```
date() {
    # Usage: date "format"
    # See: 'man strftime' for format.
    printf "%($1)T\\n" "-1"
}
```

Example Usage:

```
# Using above function.
$ date "%a %d %b  - %l:%M %p"
Fri 15 Jun  - 10:00 AM

# Using printf directly.
$ printf '%(%a %d %b  - %l:%M %p)T\n' "-1"
Fri 15 Jun  - 10:00 AM

# Assigning a variable using printf.
$ printf -v date '%(%a %d %b  - %l:%M %p)T\n' '-1'
$ printf '%s\n' "$date"
Fri 15 Jun  - 10:00 AM
```

Get the username of the current user

CAVEAT: Requires bash 4.4+

```
$ : \\u
# Expand the parameter as if it were a prompt string.
$ printf '%s\n' "${_@P}"
black
```

Generate a UUID V4

CAVEAT: The generated value is not cryptographically secure.

Example Function:

```
uuid() {
    # Usage: uuid
```

```
    C="89ab"

    for ((N=0;N<16;++N)); do
        B="$((RANDOM%256))"

        case "$N" in
            6)  printf '4%x' "$((B%16))" ;;
            8)  printf '%c%x' "${C:$RANDOM%${#C}:1}" "$((B%16))" ;;

            3|5|7|9)
                printf '%02x-' "$B"
            ;;

            *)
                printf '%02x' "$B"
            ;;
        esac
    done

    printf '\n'
}
```

Example Usage:

```
$ uuid
d5b6c731-1310-4c24-9fe3-55d556d44374
```

Progress bars

This is a simple way of drawing progress bars without needing a for loop in the function itself.

Example Function:

```
bar() {
    # Usage: bar 1 10
    #            ^----- Elapsed Percentage (0-100).
    #               ^-- Total length in chars.
    ((elapsed=$1*$2/100))

    # Create the bar with spaces.
    printf -v prog  "%${elapsed}s"
    printf -v total "%$(($2-elapsed))s"

    printf '%s\r' "[${prog// /-}${total}]"
}
```

Example Usage:

```
for ((i=0;i<=100;i++)); do
    # Pure bash micro sleeps (for the example).
    (:;:) && (:;:) && (:;:) && (:;:) && (:;:)
```

```
    # Print the bar.
    bar "$i" "10"
done

printf '\n'
```

Get the list of functions in a script

```
get_functions() {
    # Usage: get_functions
    IFS=$'\n' read -d "" -ra functions < <(declare -F)
    printf '%s\n' "${functions[@]//declare -f }"
}
```

Bypass shell aliases

```
# alias
ls

# command
# shellcheck disable=SC1001
\ls
```

Bypass shell functions

```
# function
ls

# command
command ls
```

Run a command in the background

This will run the given command and keep it running, even after the terminal or SSH connection is terminated. All output is ignored.

```
bkr() {
    (nohup "$@" &>/dev/null &)
}

bkr ./some_script.sh # some_script.sh is now running in the background
```

Capture the return value of a function without command substitution

CAVEAT: Requires bash 4+

This uses local namerefs to avoid using `var=$(some_func)` style command substitution for function output capture.

```
to_upper() {
  local -n ptr=${1}

  ptr=${ptr^^}
}

foo="bar"
to_upper foo
printf "%s\n" "${foo}" # BAR
```